Manual for using the quartet pattern counter Python and C++ library, Quartet-Pattern-Counter version 1.5

By Dr. Christoph Mayer, Nikita Kulikov, Iker Irisarri Leibniz Institute for the Analysis of Biodiversity Change, Bonn/Hamburg

What is the quartet pattern counter library

The quartet pattern counter is a C++ library which can be called form C++ as well as from Python. Its purpose is to determine from a multiple sequence alignment all or selected site pattern frequencies of quartets of sequences.

It can be used in two modes:
(i) Determine site pattern frequencies for all possible combinations of four sequences found in the input file.
(ii) Determine site pattern frequencies for all possible quartets that can be generated from the four index lists. Example: When specifying the lists `[3], [1,4], [0], [7,9]` the site pattern frequencies will be determined for all quartets that can be combined from these list, i.e.: `(3,1,0,7) (3,1,0,9) (3,4,0,7) (3,4,0,9)`, where the numbers are the zero based indices of the sequences in the input file.

The quartet pattern counter library was developed to determine site pattern frequencies for the PhyloAI program (formerly called DeepNNPhylogeny).

Quartet pattern frequencies are returned in a defined order. Furthermore, all quartet pattern returning functions have versions that return pattern lists along with the frequencies.

Requirements and Installation

- When compiling the Quartet Pattern library with your C/C++ program, nothing needs to be installed. All `*.cpp` and `*.hpp` files are needed except for the `QP_python_bindings.cpp` file.
- When using the Quartet Pattern module as a Python library, you need the `QuartetPatternCounterModule.cpython-*.so` file , compiled for your operating system in the same directory as your Python file that calls this module.
- In your Python file you need to load the module with: `input import QuartetPatternCounterModule`

If you do not have `QuartetPatternCounterModule.cpython-*.so` library file for your operating system yet, you can compile it as follows:

- Install the pybind11, numpy and setuptools packages in a separate environment and activate this environment.

- In the source code folder, type
  `make`
  This calls setup.py which compiles the library.

Using the module

A python program using the Quartet Pattern counter requires the following imports:

```
import QuartetPatternCounterModule
import numpy as np
```

In order to compute site pattern frequencies, create an object of type `QuartetPatternCounter` by calling the constructor.

(A) If you want to obtain pattern frequencies for all quartets of sequences in our input file, use the constructor. Note, that the number of possible quartets is large beyond

```
QP_obj = QuartetPatternCounter(filename, dataType, numThreads, verbose)
```

where datatype can be 'D' (DNA) or 'P' (Protein).
Higher values of verbose result in more runtime notifications (verbose >=5) and debug information (verbose>=10).

(B) If you want to obtain site patterns only for specific quartets created from combinations of indices in four arrays, use the constructor:

```
arr1 = np.array([3])    # The datatype of the numpy arrays is int32.
arr2 = np.array([1,4])
arr3 = np.array([0])
arr4 = np.array([7,9])
QP_obj = QuartetPatternCounterModule.QuartetPatternCounter(filename, arr1,
arr2, arr3, arr4, dataType, numThreads, verbose)
```

This will till the quartet pattern counter to return pattern frequencies for the 4 quartets: `(3,1,0,7) (3,1,0,9) (3,4,0,7) (3,4,0,9)`.

The two modes are distinguished by how the the `QuartetPatternCounter` is constructed. Retrieving site pattern frequencies works identical for both modes.

How pattern frequencies are returned

The most efficient way to exchange data between the module and the Python program is to not copy any data and not save any data to the hard drive. The quartet pattern counter does the following. The Python program allocates memory for the site pattern frequencies using 2D numpy arrays:

```
batch_size = 4000
freqs_D  = np.empty((batch_size, 256), dtype=np.float32)
freqs_P  = np.empty((batch_size, 160000), dtype=np.float32)
lengths  = np.empty((batch_size), dtype=np.uint32)
quartets = np.empty((batch_size, 4), dtype=np.uint16)
```

`freqs_D` is used for nucleotide sequences, `freqs_P` for amino acid (protein) sequences and the `lengths` is used if you want to retrieve the effective alignment length in the presence of gaps in the sequences. The numpy arrays are filled by the QuartetPattern module, every time when calling one of the following methods of the quartet pattern objects.

If the `QP_obj` was constructed with one of the two constructors shown above, the next batch of site pattern can be retrieved using

```
# Without getting the quartets:
N = QP_obj.next_quartet_pattern_frequencies(freqs_D, quartets)
N = QP_obj.next_quartet_pattern_frequencies_with_lengths(freqs_D,lengths)
```

The return value `N` is the actual number of quartets for which information has been written to the array. Therefore, the numpy arrays `freqs_D[0] to freqs_D[N-1]` contain site pattern frequencies for the next `N` sites.

The order of the patterns is well defined and explained below. For convenience, the pattern order can also be retrieved for each batch as follows:

```
# With getting the quartets:
N =
QP_obj.next_quartet_pattern_frequencies_with_patterns(freqs_D,quartets)
N =
QP_obj.next_quartet_pattern_frequencies_with_lengths_with_patterns(freqs_D
,
                                                                   lengths,
quartets)
```

Order of the pattern returned

If site pattern frequencies are determined for all quartets their order is:

```
number_of_sequences = 20
index_of_sequence = range(0,number_of_sequences)
for i in index_of_sequence:
  for j in index_of_sequence:
    for k in index_of_sequence:
      for l in index_of_sequence:
        print(i,j,k,l)
```

If four sets of indices are specified, the order of the patterns is:

```
for a1 in arr1:
  for a2 in arr2:
    for a3 in arr3:
      for a4 in arr4:
        print(f"({a1},{a2},{a3},{a4}),")
```

How gaps are treated

For each quartet of sequences, all site patterns containing gaps are removed. All other patterns are counted and relative frequencies are determined.

Examples

Determining the site pattern frequencies for all quartets

```
import QuartetPatternCounterModule
import numpy as np

filename = "input.fas"
numThreads = 1
dataType = 'D'
verbose = 20
batchsize = 2000

QuartetPatternCounter(filename, dataType, numThreads, verbose)
freqs   = np.empty((batch_size, 256), dtype=np.float32)
lengths = np.empty((batch_size), dtype=np.uint32)

N = QPcounter_fc.next_quartet_pattern_frequencies_with_lengths(freqs,
lengths)

  batch = 0
  while N!=0:
    print("Number of quartets determined: ", N)
    for i in range(N):
      print("Combination: ", i+1+batch*batch_size, " length: ",
            length_array[i+batch*batch_size])
      print_freqs(freq_array[i])
    batch +=1
    N = QPcounter_fc.next_quartet_pattern_frequencies(freq_array)

# OR with patterns:
  batch_size = 5000
  freq_array = np.empty((batch_size, 256), dtype=np.float32)
  length_array = np.empty((batch_size), dtype=np.uint32)
```

```
  pattern_array = np.empty((batch_size, 4), dtype=np.uint16)

  N =
QPcounter_fc.next_quartet_pattern_frequencies_with_lengths_with_patterns(
                                        freq_array, length_array,
pattern_array)
  batch = 0
  while N!=0:
    print("Number of quartets determined: ", N)
    for i in range(N):
      print("Combination: ", i+1+batch*batch_size, " length: ",
            length_array[i+batch*batch_size], " pattern: ",
pattern_array[i])
      print_freqs(freq_array[i])
    batch +=1
    N =
QPcounter_fc.next_quartet_pattern_frequencies_with_lengths_with_patterns(
            freq_array, length_array, pattern_array)
```

Determining the site pattern frequencies for using four index sets

```
import QuartetPatternCounterModule
import numpy as np

filename = "input.fas"
numThreads = 1
dataType = 'D'
verbose = 20
batchsize = 2000

arr1 = np.array([3])
arr2 = np.array([1,4])
arr3 = np.array([0])
arr4 = np.array([7,9])
QPcounter_fc = QuartetPatternCounterModule.QuartetPatternCounter(filename,
arr1,                                  arr2, arr3, arr4, dataType,
numThreads, verbose)
freqs   = np.empty((batch_size, 256), dtype=np.float32)
lengths = np.empty((batch_size), dtype=np.uint32)

N = QPcounter_fc.next_quartet_pattern_frequencies_with_lengths(freqs,
lengths)

batch = 0
while N!=0:
  print("Number of quartets determined: ", N)
```

```
  for i in range(N):
    print("Combination: ", i+1+batch*batch_size, " length: ",
          length_array[i+batch*batch_size])
    print_freqs(freq_array[i])
  batch +=1
  N = QPcounter_fc.next_quartet_pattern_frequencies(freq_array)

# OR with patterns:

N =
QPcounter_fc.next_quartet_pattern_frequencies_with_lengths_with_patterns(
                                        freq_array, length_array,
pattern_array)
batch = 0
while N!=0:
  print("Number of quartets determined: ", N)
  for i in range(N):
    print("Combination: ", i+1+batch*batch_size, " length: ",
          length_array[i+batch*batch_size], " pattern: ",
pattern_array[i])
    print_freqs(freq_array[i])
  batch +=1
  N =
QPcounter_fc.next_quartet_pattern_frequencies_with_lengths_with_patterns(
          freq_array, length_array, pattern_array)
```

Note how the two versions only differ in defining the four index arrays and calling a different constructor which obtains the four index arrays as parameters.